

# Python Idioms Reference

CodeSignal Assessment Prep

## Quick Reference — Open These Tabs First

---

- [docs.python.org/3/library/functions.html](https://docs.python.org/3/library/functions.html) — Built-ins: len, sorted, min, max, sum, enumerate, zip, range, isinstance, any, all, next
- [docs.python.org/3/library/stdtypes.html#text-sequence-type-str](https://docs.python.org/3/library/stdtypes.html#text-sequence-type-str) — String methods: split, join, strip, find, replace, startswith
- [docs.python.org/3/library/stdtypes.html#mapping-types-dict](https://docs.python.org/3/library/stdtypes.html#mapping-types-dict) — Dict methods: get, items, keys, values, setdefault, pop
- [docs.python.org/3/library/threading.html](https://docs.python.org/3/library/threading.html) — Thread, Lock, start, join
- [docs.python.org/3/library/queue.html](https://docs.python.org/3/library/queue.html) — Queue, put, get, Empty, task\_done
- [docs.python.org/3/library/asyncio.html](https://docs.python.org/3/library/asyncio.html) — async def, await, gather, run, sleep
- [docs.python.org/3/library/collections.html](https://docs.python.org/3/library/collections.html) — defaultdict, Counter, deque
- [docs.python.org/3/library/unittest.html](https://docs.python.org/3/library/unittest.html) — assertEquals, assertRaises, setUp, assertIsNone, assertTrue
- [docs.python.org/3/library/stdtypes.html#list](https://docs.python.org/3/library/stdtypes.html#list) — List methods: append, extend, pop, remove, insert, sort

## Dict Patterns [docs](#)

---

```
d.get(key, default)          # safe lookup, no KeyError
d.get(key, 0) + 1            # increment with default -- memorize this
d.setdefault(key, []).append(x) # get or create, then use

# Running counter (word count pattern)
counts = {}
for x in items:
    counts[x] = counts.get(x, 0) + 1

# Existence check
if key in d:                 # True if key exists
if key not in d:             # True if key is absent -- prefer over "not key in d"
```

## Sorting with Keys [docs](#)

---

```
sorted(items, key=lambda x: x["score"])          # sort by field
sorted(items, key=lambda x: (-x["score"], x["name"])) # desc score, asc
                                                    # name (tie-break)
sorted(items, key=lambda x: x[1], reverse=True)    # sort list of
                                                    # tuples by second element
lst.sort()          # sorts in place, returns None
sorted(lst)         # returns new list, original unchanged

# min/max without sorting the whole list
min(items, key=lambda x: x["id"])                # item with lowest id -- O(n), cleaner
                                                    # than sorted()[0]
max(items, key=lambda x: x["score"])              # item with highest score
```

## Unpacking

---

```
a, b = b, a          # swap without temp variable
first, *rest = [1, 2, 3, 4] # first=1, rest=[2,3,4]
*init, last = [1, 2, 3, 4]  # init=[1,2,3], last=4
```

```

a, b = some_tuple           # unpack a 2-tuple
for i, x in enumerate(lst): # index + value simultaneously
for a, b in zip(list1, list2): # parallel iteration

# Tuple unpacking in loops (common in CodeSignal action lists)
for action_type, *args in actions: # action_type = first element, rest in
    args
    if action_type == "post":
        author, content = args

```

## Comprehensions

```

[f(x) for x in lst if condition(x)] # filter + transform
{k: v for k, v in d.items() if v > 0} # filter a dict
{k: [] for k in keys} # initialize dict of lists
[[0]*cols for _ in range(rows)] # 2D list (NOT [[0]*cols]*rows --
    shared ref bug)

# Flattening one level
[x for sublist in nested for x in sublist]

# Set comprehension -- get unique values
{item["author"] for item in books}

```

## String Operations [docs](#)

```

", ".join(["a", "b", "c"]) # -> "a, b, c"
"hello world".split() # -> ["hello", "world"] (any whitespace)
"a:b:c".split(":") # -> ["a", "b", "c"] (specific delimiter)
" hello ".strip() # -> "hello" (strips both ends)
" hello ".lstrip() # -> "hello " (left only)

# f-string format specs
f"{value:.2f}" # float with 2 decimal places: "3.14"
f"{value:>10}" # right-align in 10-char field: " 3.14"
f"{value:<12}" # left-align in 12-char field: "3.14 "
f"{value:03d}" # zero-pad integer: "007"
f"{value!r}" # repr (adds quotes) -- useful for debugging

# Parsing a structured line (common in assessments)
fn_id, event, timestamp = log_line.split(":")
name, score, category = line.strip().split(":")

# Skip blank lines
lines = [l for l in text.split("\n") if l.strip()]

```

## Control Flow Idioms

```

x = value if condition else default # ternary
any(x > 0 for x in lst) # True if any element satisfies
all(x > 0 for x in lst) # True if all elements satisfy
next((x for x in lst if x > 0), None) # first match or None (safe)

# Truthiness -- prefer over len() checks
if lst: # True if list is non-empty
if not lst: # True if list is empty
if d: # True if dict is non-empty

```

```
if queued_jobs:    # cleaner than if len(queued_jobs) > 0:
```

## Working with Indices

---

```
lst[-1]            # last element
lst[-2]            # second to last
lst[::-1]          # reversed copy
lst[i:j]           # slice i to j (exclusive)
(n-1)//2           # leftmost middle index (use for median -- verified against
                    # examples)
n//2               # rightmost middle index for even n
```

## Collections (standard library) [docs](#)

---

```
from collections import defaultdict
d = defaultdict(int)      # missing keys return 0
d = defaultdict(list)     # missing keys return []
d["new_key"].append(x)    # no KeyError even if key is absent

from collections import Counter
c = Counter([1, 1, 2, 3]) # -> {1:2, 2:1, 3:1}
c.most_common(3)          # top 3 by count as list of (value, count) tuples

from collections import deque
q = deque()
q.appendleft(x)           # O(1) prepend (list.insert(0,x) is O(n))
q.popleft()               # O(1) pop from front
```

## Class Design Patterns

---

### Manager class – standard CodeSignal pattern

```
class ManagerImpl:
    def __init__(self):
        self.items = {}          # id -> item dict -- O(1) lookup, not a list
        self.next_id = 1
        self.lock = threading.Lock() # add lock immediately even if not yet
                                     # needed

    def add_item(self, name):
        item_id = self.next_id
        self.items[item_id] = {
            "id": item_id, "name": name, "status": "pending",
            "total_ops": 0        # add ALL fields you'll need for later levels
        }
        self.next_id += 1
        return item_id

    def get_item(self, item_id):
        return self.items.get(item_id) # returns None if not found -- use this
```

### Status transition pattern

```
def start(self, item_id):
    if item_id not in self.items:
        raise ValueError("not found")
    if self.items[item_id]["status"] != "pending":
```

```

        raise ValueError(f"expected pending, got
                           {self.items[item_id]['status']}")
    self.items[item_id]["status"] = "active"

```

### Aggregation – iterating stored items

```

# Total copies across all books
total = sum(b["copies"] for b in self.library.values())

# Unique users (flattening a list-of-lists field)
all_users = {u for item in self.items.values() for u in item["checked_out_by"]}

# Top n by field, with tie-breaking
top = sorted(self.items.values(),
              key=lambda x: (-x["total_checkouts"], x["title"]))[:n]
result = [{"id": b["id"], "title": b["title"],
           "total_checkouts": b["total_checkouts"]} for b in top]

```

## Threading docs

---

```

import threading

# Basic thread -- start all, then join all (never join inside start loop)
threads = []
for f in funcs:
    t = threading.Thread(target=f)           # target is function object, no ()
    t.start()
    threads.append(t)
for t in threads:
    t.join()                                # waits for all to finish

# Thread with arguments
t = threading.Thread(target=worker, args=("Alice",)) # args must be a tuple
t = threading.Thread(target=worker, kwargs={"name": "Alice"})

# Lock -- prevents race conditions on shared state
lock = threading.Lock()
with lock:                                   # only one thread runs this block
    at a time
    shared_value += 1

# CAUTION: never hold a lock while calling t.join() on threads that need that
lock
# That is a deadlock.

```

### Worker pool – competing for jobs from shared state

```

# Pattern: lock wraps state ACCESS only -- work happens outside the lock
def run_parallel(self, n_workers):
    def worker():
        while True:
            with self.lock:
                job = self.start_next()    # atomically grab AND mark the job
            if job is None:
                break
            self.finish(job["id"])         # work outside lock = true
    parallelism

```

```

threads = [threading.Thread(target=worker) for _ in range(n_workers)]
for t in threads: t.start()
for t in threads: t.join()

```

*# KEY: start\_next() must atomically find AND change status.  
 # If you separate "find" and "mark" into two locked calls, two workers can  
 # both see the same job between the two calls -- race condition.*

### Ordered results from parallel tasks

```

# Pre-allocate results list -- write by index, not append
def parallel_map(tasks, n_workers):
    results = [None] * len(tasks)    # pre-allocate by length
    lock = threading.Lock()
    next_i = 0

    def worker():
        nonlocal next_i              # required to assign to enclosing variable
        while True:
            with lock:
                i = next_i
                if i >= len(tasks):
                    break
                next_i += 1            # grab index under lock (atomic
                                     # check+increment)
            results[i] = tasks[i]()    # write to pre-allocated slot -- no lock
                                     # needed
                                     # (each worker writes to a different index)

    threads = [threading.Thread(target=worker) for _ in range(n_workers)]
    for t in threads: t.start()
    for t in threads: t.join()
    return results

# nonlocal: required when an inner function ASSIGNS to an outer variable.
# Reading an outer variable is fine without it; assigning triggers
# UnboundLocalError.
def outer():
    count = 0
    def inner():
        nonlocal count    # without this: UnboundLocalError on count += 1
        count += 1
    inner()
    return count           # 1

```

### Thread-safe counter

```

class ThreadSafeCounter:
    def __init__(self):
        self._count = 0
        self._lock = threading.Lock()

    def increment(self):
        with self._lock:
            self._count += 1

    @property

```

```
def value(self):
    with self._lock:
        return self._count
```

## Queue (thread-safe communication) [docs](#)

```
import queue

q = queue.Queue()           # thread-safe FIFO -- no lock needed for get/put
q.put(item)                 # add item
item = q.get(block=False)   # raises queue.Empty if empty -- use in try/except
q.task_done()              # signal that a gotten item has been processed
q.join()                   # block until all items have been task_done()

# Worker pool with queue.Queue -- cleaner than manual lock when tasks are
# independent
def process_with_workers(items, fn, n_workers):
    q = queue.Queue()
    results = {}
    lock = threading.Lock()  # queue is thread-safe; results dict is NOT

    for item in items:
        q.put(item)

    def worker():
        while True:
            try:
                item = q.get(block=False)  # raises queue.Empty when done
            except queue.Empty:
                break
            value = fn(item)
            with lock:
                results[item] = value      # protect the results dict

    threads = [threading.Thread(target=worker) for _ in range(n_workers)]
    for t in threads: t.start()
    for t in threads: t.join()
    return results

# Use queue.Queue when distributing plain work items.
# Use manual lock when "get next item" has side effects (e.g. marking a job as
# running).
```

## asyncio [docs](#)

```
import asyncio

# Declare a coroutine
async def fetch(url):
    await asyncio.sleep(1)  # await suspends this coroutine, lets others run
    return url

# Run concurrently -- equivalent of threading run_parallel
async def main():
    results = await asyncio.gather(fetch("a"), fetch("b"), fetch("c"))
    return results
```

```

asyncio.run(main())           # entry point -- starts the event loop, call
    once

# Gather from a list (unpack with *)
coros = [fetch(url) for url in urls]
results = await asyncio.gather(*coros)

# asyncio.sleep vs time.sleep
await asyncio.sleep(1)        # yields control to event loop (correct)
time.sleep(1)                 # BLOCKS entire event loop (wrong in async code)

```

## Threading vs asyncio

|                   | threading                               | asyncio                                     |
|-------------------|-----------------------------------------|---------------------------------------------|
| Syntax            | regular functions                       | async def / await                           |
| Parallelism       | OS threads (true concurrency)           | single thread, cooperative                  |
| Shared state      | needs Lock                              | safer by default (one thing runs at a time) |
| Best for          | any blocking work, simpler mental model | high-concurrency I/O                        |
| Worker loop       | natural while True                      | needs await to yield                        |
| CodeSignal advice | <b>prefer this</b>                      | use if spec requires it                     |

## Exception Handling [docs](#)

```

try:
    result = risky_operation()
except KeyError:
    result = default
except (TypeError, ValueError) as e:
    raise RuntimeError("bad input") from e
except Exception:           # catches all normal errors (not
    KeyboardInterrupt)
    return None
finally:
    cleanup()                # always runs

# Raise with message
raise ValueError("Cannot divide by zero")
raise TypeError(f"Expected int, got {type(x).__name__}")

# isinstance for type checking
isinstance(x, (int, float))  # True if x is int or float -- not type(x) ==
    int

```

## In-Memory Key-Value Store Patterns

*Basic structure with TTL (time-to-live) [time docs](#)*

```

import time

class KVStore:
    def __init__(self):
        self.data = {}      # key -> value
        self.expires = {}   # key -> expiry timestamp (float)
                                # two separate dicts -- cleaner than nesting

    def set(self, key, value):

```

```

        self.data[key] = value
        self.expires.pop(key, None)    # clear any existing TTL

    def set_with_ttl(self, key, value, ttl):
        self.data[key] = value
        self.expires[key] = time.time() + ttl    # store absolute expiry time

    def _expired(self, key):
        if key not in self.expires:
            return False
        return time.time() >= self.expires[key]    # >= not >

    def get(self, key):
        if key not in self.data or self._expired(key):
            return None
        return self.data[key]

    def delete(self, key):
        if key in self.data and not self._expired(key):
            del self.data[key]
            self.expires.pop(key, None)
            return True
        return False

# Prefix scan -- return all live keys matching a prefix
def scan(self, prefix):
    return {k: v for k, v in self.data.items()
            if k.startswith(prefix) and not self._expired(k)}

```

### Transactions (begin / commit / rollback)

```

_DELETED = object()    # sentinel object -- distinct from any real value

class KVStore:
    def __init__(self):
        self.data = {}
        self.buffer = None    # None = no active transaction; dict = transaction
                                open

    def begin(self):
        self.buffer = {}

    def commit(self):
        if self.buffer is None:
            return False
        for key, value in self.buffer.items():
            if value is _DELETED:    # buffered deletion
                self.data.pop(key, None)
            else:
                self.data[key] = value
        self.buffer = None
        return True

    def rollback(self):
        if self.buffer is None:
            return False
        self.buffer = None    # discard without applying
        return True

```

```

def set(self, key, value):
    if self.buffer is not None:
        self.buffer[key] = value      # buffer it -- don't touch data yet
    else:
        self.data[key] = value

def get(self, key):
    if self.buffer is not None and key in self.buffer:
        v = self.buffer[key]
        return None if v is _DELETED else v    # respect buffered deletes
    return self.data.get(key)

def delete(self, key):
    if self.buffer is not None:
        if key in self.buffer or key in self.data:
            self.buffer[key] = _DELETED
            return True
        return False
    if key in self.data:
        del self.data[key]
        return True
    return False

# Pattern summary:
# self.buffer = None    -> no transaction
# self.buffer = {}      -> transaction open; writes go here
# _DELETED sentinel    -> records a deletion inside the buffer
# get() checks buffer first, then data -> reads see uncommitted writes

```

## Stack Simulation Pattern

---

```

# Pattern: prev_time tracks when current top-of-stack last started or resumed.
# Used for problems involving nested function calls, intervals, or nested
# events.
stack = []
prev_time = 0

for log in logs:
    fn_id, event, timestamp = log.split(":")
    fn_id, timestamp = int(fn_id), int(timestamp)

    if event == "start":
        if stack:
            exclusive[stack[-1]] += timestamp - prev_time    # time before this
                                                                call
            stack.append(fn_id)
            prev_time = timestamp
        else: # end
            exclusive[stack[-1]] += timestamp - prev_time + 1 # +1: end is
                                                                inclusive
            stack.pop()
            prev_time = timestamp + 1                          # resume at next
                                                                unit

```

## R-to-Python Gotchas

---

```

# Indexing: Python is 0-indexed
lst[0]          # first element (not lst[1])
lst[-1]         # last element (not lst[length])

# Integer division
10 / 3          # -> 3.333... (float, even for ints)
10 // 3         # -> 3 (floor division)

# None check
if x is None:   # correct
if x == None:   # works but wrong style

# Mutability: Python passes lists by reference
def f(lst):
    lst.append(1) # modifies the ORIGINAL list

# append returns None -- don't assign it
result = lst.append(x) # result is None -- common bug

# Logical operators
not x          # not !x
and            # not &&
or             # not ||

# No paste() -- use f-strings
f"hello {name}, you are {age} years old"

# range vs R's seq
range(n)        # 0, 1, ..., n-1 (not 1 to n)
range(a, b)     # a, a+1, ..., b-1

# Shadowing builtins -- don't use these as variable names
# id, list, dict, set, type, input, range, min, max, sum, len, zip

```

## Assessment Strategy

- Read the **full spec** before writing any code – all levels.
- Open the test file first – tests are the spec, not the description.
- **Sketch `__init__` before touching code:** what data structures, what fields does Level 4+ require? Add all fields upfront.
- Use **dict** keyed by **id**, not a list –  $O(1)$  lookup, simpler methods.
- Start with Level 1, pass all tests before moving on.
- Run tests early and often (**bash** `run_single_test.sh "<name>"` for one test).
- Don't optimize. Write correct first.
- For concurrency: use **threading** with **Lock** unless **asyncio** is explicitly required.
- If stuck on a level, move on – partial credit beats zero.
- Leave 5 minutes to re-run all tests at the end.
- Language docs and Google (AI Overview only) are allowed.